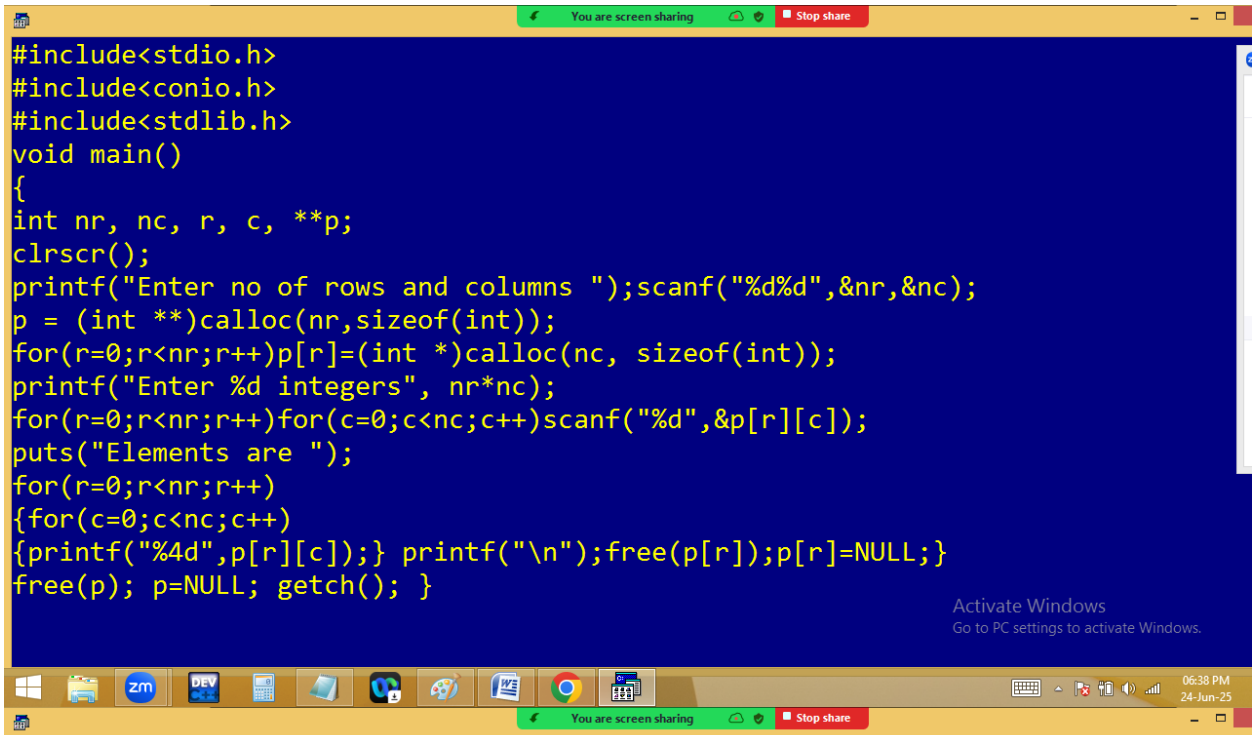
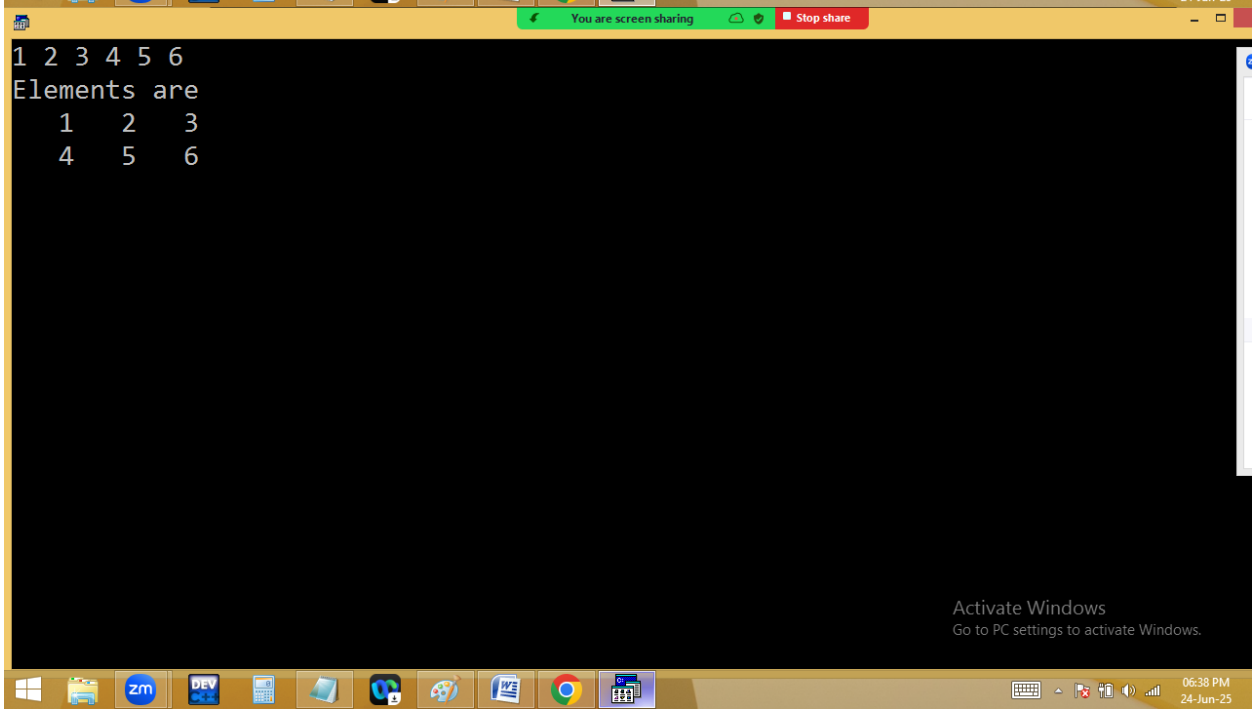
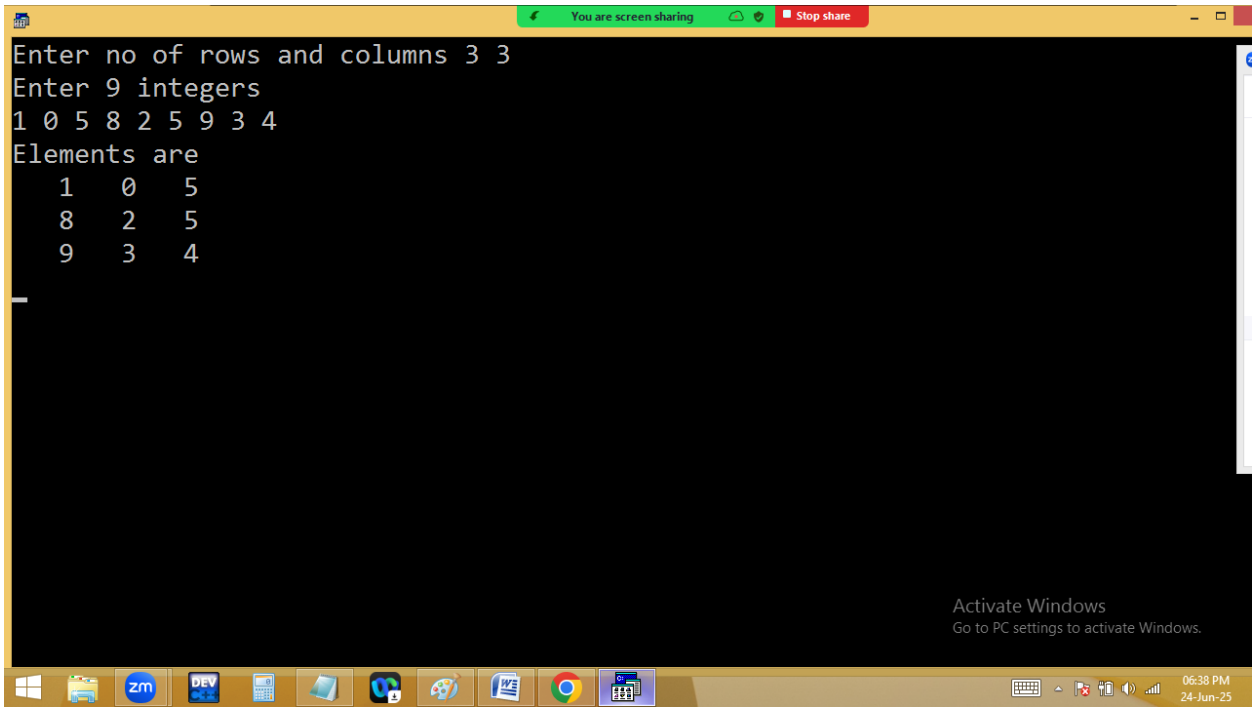
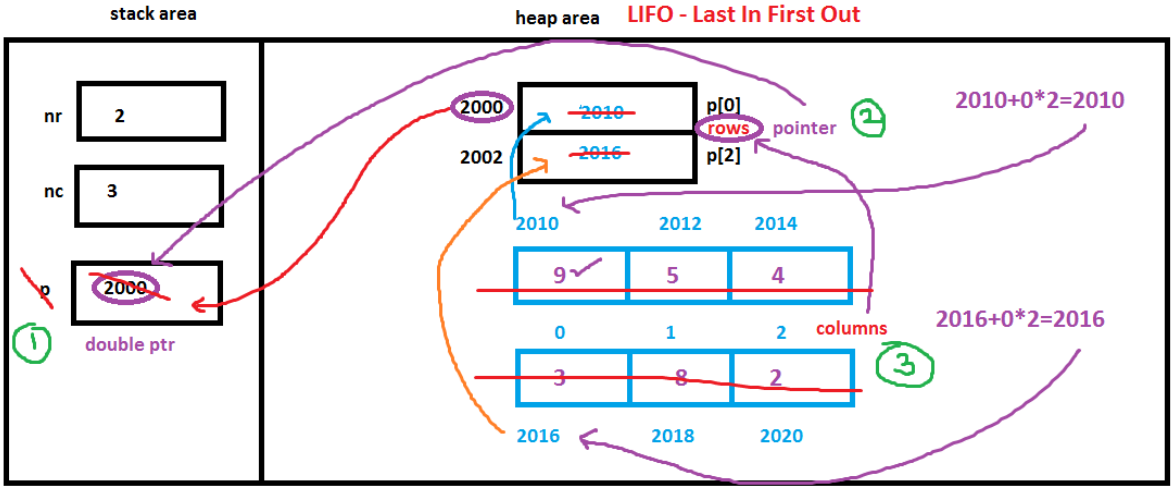
Dynamic multi-dimensional array:

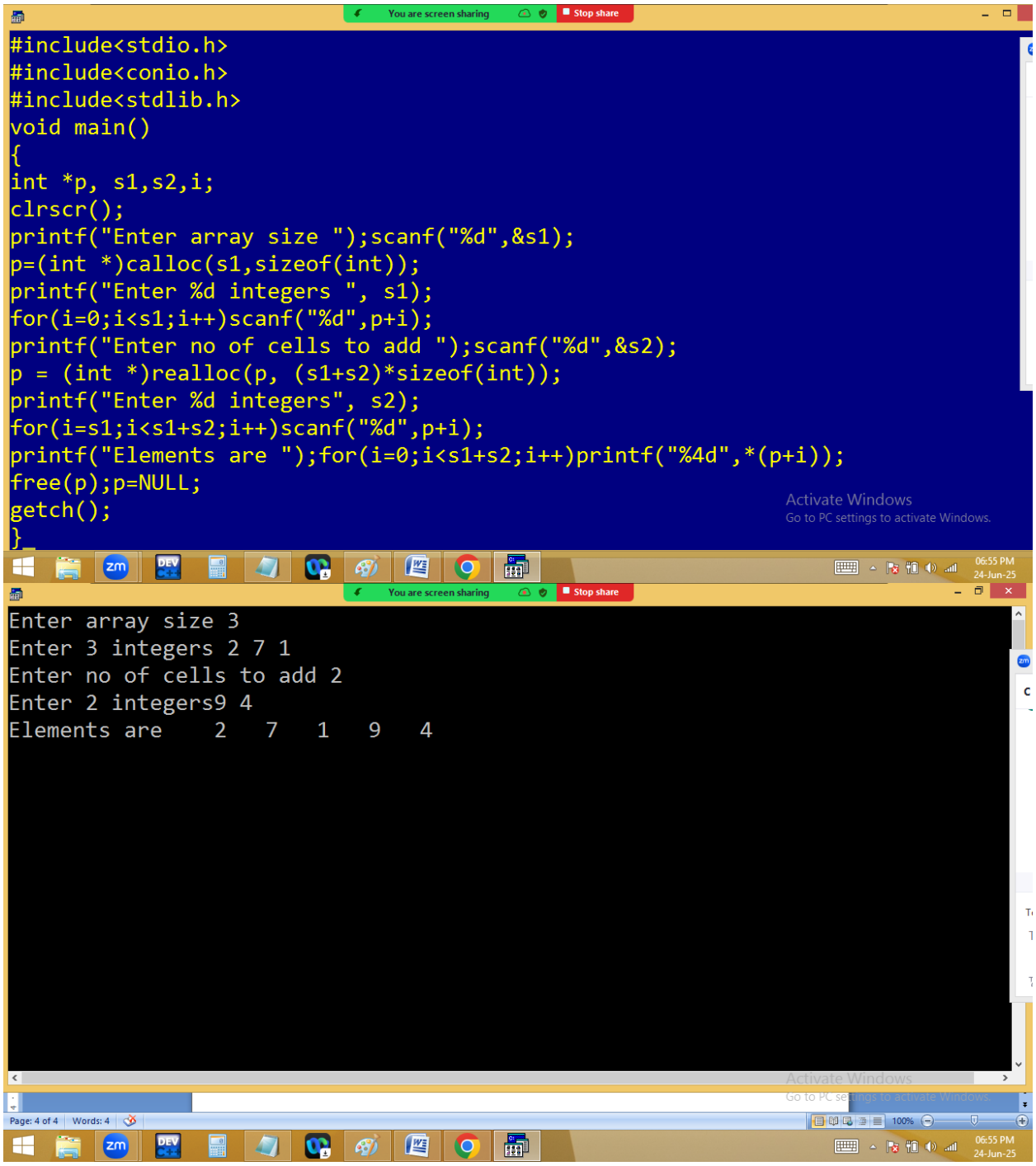

You are screen sharing
Stop share
#include<stdio.h>
#include<conio.h>
#include<stdlib.h>
void main()
{
int nr, nc, r, c, **p;
clrscr();
printf("Enter no of rows and columns ");scanf("%d%d",&nr,&nc);
p = (int **)calloc(nr,sizeof(int));
for(r=0;r<nr;r++)p[r]=(int *)calloc(nc, sizeof(int));
printf("Enter %d integers", nr*nc);
for(r=0;r<nr;r++)for(c=0;c<nc;c++)scanf("%d",&p[r][c]);
puts("Elements are ");
for(r=0;r<nr;r++)
{for(c=0;c<nc;c++)
{printf("%4d",p[r][c]);} printf("\n");free(p[r]);p[r]=NULL;}
free(p); p=NULL; getch(); }
Activate Windows
Go to PC settings to activate Windows.
06:38 PM
24-Jun-25


You are screen sharing
Stop share
1 2 3 4 5 6
Elements are
1 2 3
4 5 6
Activate Windows
Go to PC settings to activate Windows.
06:38 PM
24-Jun-25


You are screen sharing
Stop share
Enter no of rows and columns 3 3
Enter 9 integers
1 0 5 8 2 5 9 3 4
Elements are
1 0 5
8 2 5
9 3 4
Activate Windows
Go to PC settings to activate Windows.
06:38 PM
24-Jun-25

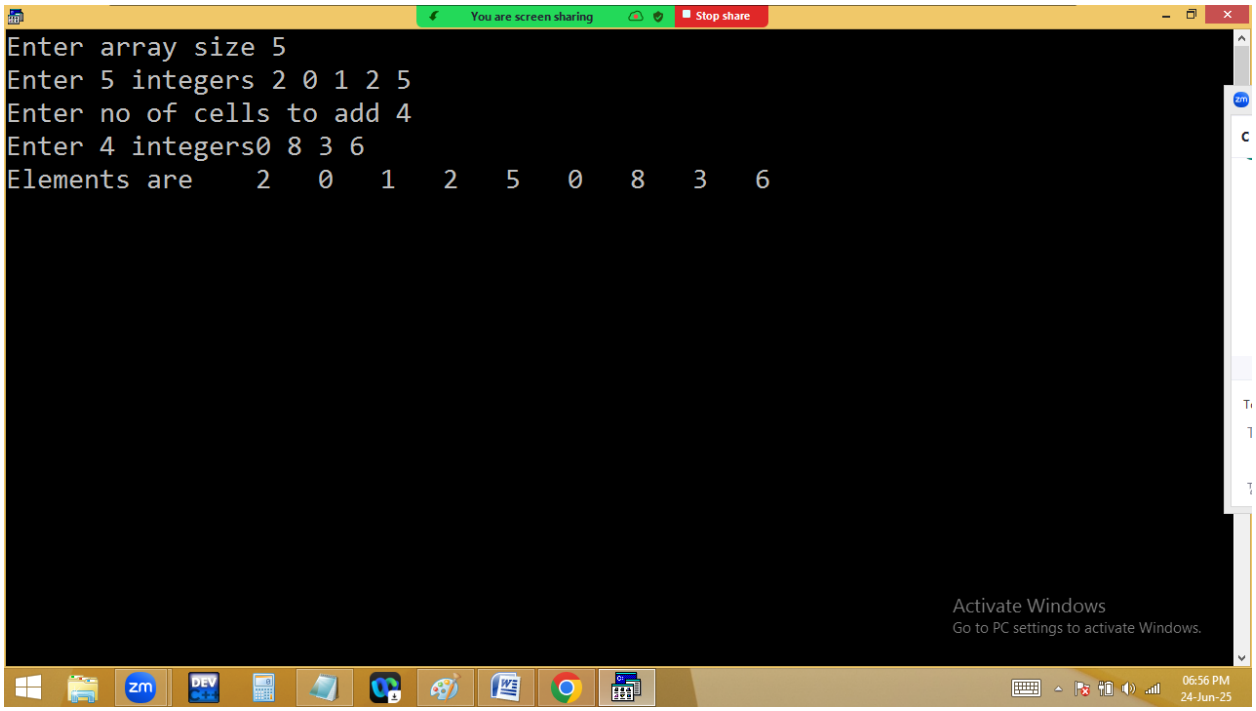

stack area
heap area
LIFO - Last In First Out
nr
2
nc
3
p
2000
double ptr
2000
2002
2010
2016
p[0]
rows
pointer
p[2]
2010+0*2=2010
2010
2012
2014
9
5
4
0
1
2
columns
3
8
2
2016+0*2=2016
2016
2018
2020

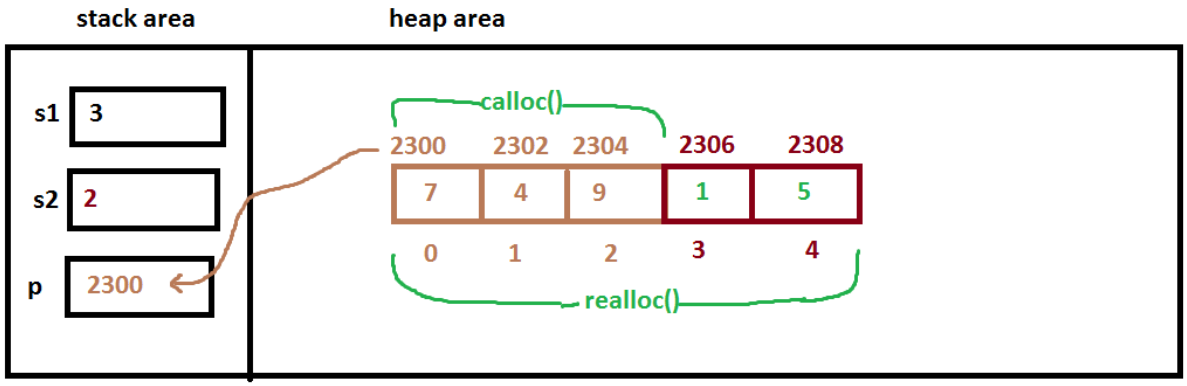
realloc():



```c
#include<stdio.h>
#include<conio.h>
#include<stdlib.h>
void main()
{
int *p, s1,s2,i;
clrscr();
printf("Enter array size ");scanf("%d",&s1);
p=(int *)calloc(s1,sizeof(int));
printf("Enter %d integers ", s1);
for(i=0;i<s1;i++)scanf("%d",p+i);
printf("Enter no of cells to add ");scanf("%d",&s2);
p = (int *)realloc(p, (s1+s2)*sizeof(int));
printf("Enter %d integers", s2);
for(i=s1;i<s1+s2;i++)scanf("%d",p+i);
printf("Elements are ");for(i=0;i<s1+s2;i++)printf("%4d",*(p+i));
free(p);p=NULL;
getch();
}
```

```
Enter array size 3
Enter 3 integers 2 7 1
Enter no of cells to add 2
Enter 2 integers9 4
Elements are    2   7   1   9   4
```


You are screen sharing
Stop share
Enter array size 5
Enter 5 integers 2 0 1 2 5
Enter no of cells to add 4
Enter 4 integers0 8 3 6
Elements are 2 0 1 2 5 0 8 3 6
Activate Windows
Go to PC settings to activate Windows.
06:56 PM
24-Jun-25


stack area
heap area
s1
3
s2
2
p
2300
calloc()
2300
2302
2304
2306
2308
7
4
9
1
5
0
1
2
3
4
realloc()

USER DEFINED FUNCTIONS

What is a function?

1. It is a small program used to do a particular task.
2. It is a small program with in another program.
3. It is a sub program or sub routine.
4. It is a procedure.
5. It is a self-contained block.
6. It is a reusable code component.
7. It is a module.

Advantages:

1. **Modularity**: Dividing program instructions into small pieces.
2. **Simplicity**: It is easy to understand the program instructions.
3. **Reusability**: Just write once, use several times.
4. **Efficiency**: performance is improved.
5. It is easy to identify the errors.

Entire ‘C’ program is collection of functions. Hence ‘C’ is a function oriented structured programming language.

We are having 2 types of functions.

1. Predefined / library / built-in functions

These are the functions provided with software and ready to use.

Eg: printf(), scanf(),…

2. **User defined functions**: They are created by the user.

Eg: sum(), prime(), factorial(),……

Every user defined function is divided into 3 parts.

1. Function declaration/ proto typing
2. Function calling
3. Function definition

Function declaration/proto typing:

Generally function declaration is conducted before or within the main(), before function calling.

It tells the compiler that we are going to use this kind of function in our program in future.

Syntax:

[return_datatype] function_name ([arguments / parameters]);

Eg:

void sum();

Function calling:

It should be conducted within the main() only.

When a function is invoked (called), the compiler will search for the matching function definition and it is available then the program execution is jumped from function calling area to the function definition area.

Linking a function call to the function definition is called **binding.** C language supports only static or compile time binding. But C++ supports static and dynamic binding[run time binding].

Without function call, a function never participates in function execution. Hence it is mandatory to call a function in a program.

Syntax:

function_name([arguments]);

Eg: sum();

Function definition:

It contains the function header and body. The function header should be matched with function declaration.

It is conducted outside the main(). If the definition is conducted before the main(), there is no need of function declaration.

Function header consists of function name and the parameters.

Function body consists of statements related to the task of function.

When a function is invoked, the program execution is shifted from function calling area (main()) to function definition area. After executing the function body, the execution is jumped to the next statement after the function call in main().

Function declaration and function definition should be identical.